

# Học Sâu cho Khoa Học Dữ liệu

## Chương 5. LSTM, GRU, Attention is all you need

Hà Minh Tuấn  
hmtuan@hcmus.edu.vn

Khoa Toán - Tin học  
Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

Ngày 7 tháng 4 năm 2026

# Nội dung bài học

1. ví dụ mở đầu
2. Long - short term memory LSTM
3. Các ví dụ
4. Code mẫu python cho LSTM
5. GRU - Gated Recurrent Units
6. Kiến trúc GRU
7. Các ví dụ GRU
8. Deep RNN
9. Bidirection RNN
10. Encoder - Decoder

# Long Short-Term Memory (LSTM)

From RNN to a better memory mechanism

Why do we need LSTM?

# Bài toán: Đoán từ tiếp theo

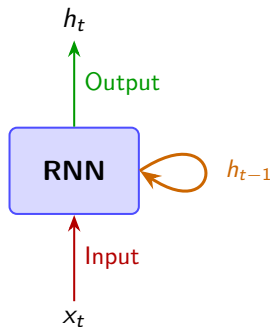
**Thử sức nào!** Hãy đoán từ còn thiếu:

- ❶ “Hôm nay trời rất \_\_\_\_\_” → **đẹp / nắng / mưa**
- ❷ “Con mèo ngồi trên \_\_\_\_\_” → **bàn / mái nhà**
- ❸ “Tôi sinh ra ở Việt Nam. Tôi lớn lên ở Hà Nội.  
Sau 20 năm, tôi vẫn nhớ quê hương. Tiếng mẹ đẻ của tôi là tiếng \_\_\_\_\_”  
→ **Việt**

## Nhận xét

- Câu 1–2: Chỉ cần nhìn **vài từ gần** là đoán được
- Câu 3: Phải nhớ thông tin từ **rất xa** (“Việt Nam”) mới đoán đúng

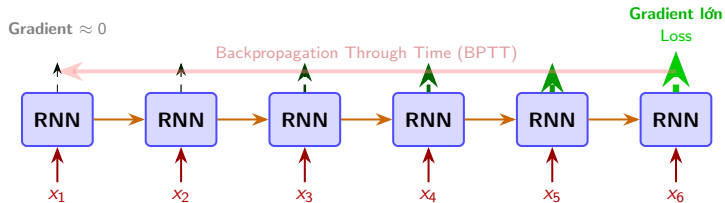
## Nhắc lại: RNN hoạt động thế nào?



Dạng cuộn (Rolled)

- Tại mỗi bước  $t$ :  $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$
- Trạng thái ẩn  $h_t$  mang thông tin từ **tất cả** các bước trước
- Nhưng thực tế có đúng vậy không?

# Vấn đề: Vanishing Gradient — Trí nhớ ngắn hạn



## Vanishing Gradient Problem

- Gradient nhân liên tiếp qua nhiều bước  $\Rightarrow$  **co lại về 0**
- Kết quả: RNN **không học được** phụ thuộc xa (long-range dependency)
- Ở câu 3 slide trước: RNN “quên” từ “Việt Nam” khi đến cuối câu

# Ý tưởng cốt lõi: Cần một “ô nhớ” thông minh

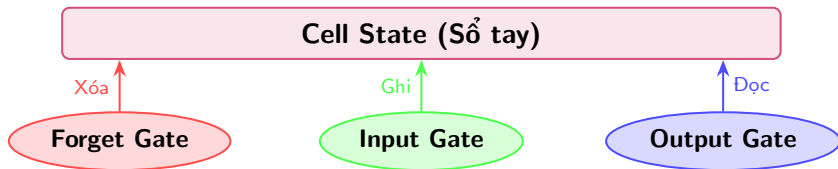
**Phép ẩn dụ:** Hãy tưởng tượng bạn đọc một cuốn sách dài.

RNN = Đọc không ghi chép

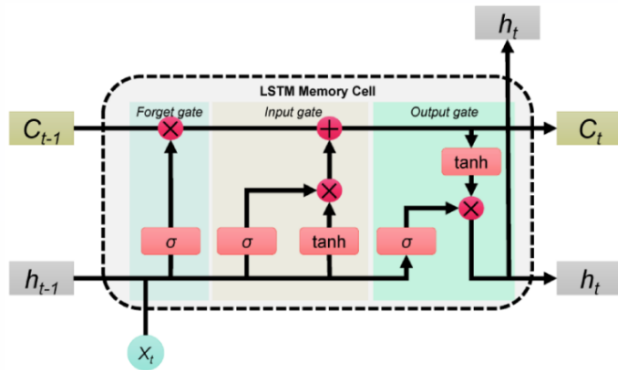
- Chỉ nhớ mang máng vài trang gần nhất
- Đến chương 10 thì quên hết chương 1
- Không kiểm soát được nhớ gì, quên gì

LSTM = Đọc có sổ tay

- Có **sổ tay** (Cell State) ghi lại điều quan trọng
- Biết **khi nào ghi thêm** (Input Gate)
- Biết **khi nào xóa bớt** (Forget Gate)
- Biết **khi nào đọc lại** (Output Gate)



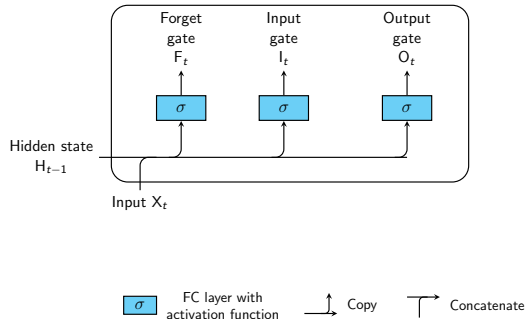
# Cấu trúc bên trong LSTM Memory Cell



LSTM networks operate similarly to RNNs but include specialized mechanisms called gates: Forget gate, input gate, and output gate. These gates regulate how information flows through the network, allowing it to retain important information over extended periods. The forget gate decides which past information to discard, the input gate controls the relevance of new information, and the output gate



# Tính toán các cổng trong LSTM



Giả sử có  $h$  hidden units, batch size  $n$ , số đầu vào  $d$ .

**Đầu vào:**

- $X_t \in \mathbb{R}^{n \times d}$ : input tại bước  $t$
- $H_{t-1} \in \mathbb{R}^{n \times h}$ : hidden state bước trước

Ba cổng ( $\in \mathbb{R}^{n \times h}$ ) được tính:

$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i)$$

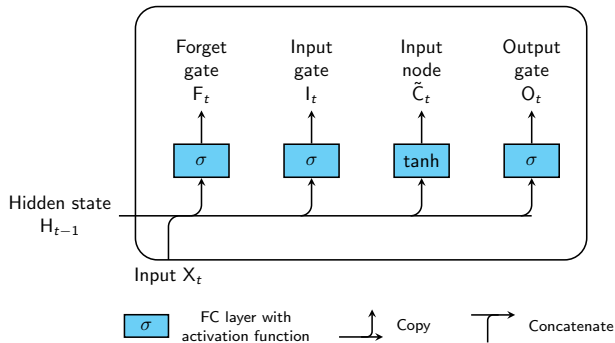
$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f)$$

$$O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o)$$

**Tham số:**

- $W_{xi}, W_{xf}, W_{xo} \in \mathbb{R}^{d \times h}$
- $W_{hi}, W_{hf}, W_{ho} \in \mathbb{R}^{h \times h}$
- $b_i, b_f, b_o \in \mathbb{R}^{1 \times h}$  (bias)

# Designing the Memory Cell (Input Node)



Thiết kế **ô nhớ** (memory cell): giới thiệu **nút đầu vào** (input node)  $\tilde{C}_t \in \mathbb{R}^{n \times h}$ .

**Đầu vào:**

- $X_t \in \mathbb{R}^{n \times d}$ : input tại bước  $t$
- $H_{t-1} \in \mathbb{R}^{n \times h}$ : hidden state bước trước

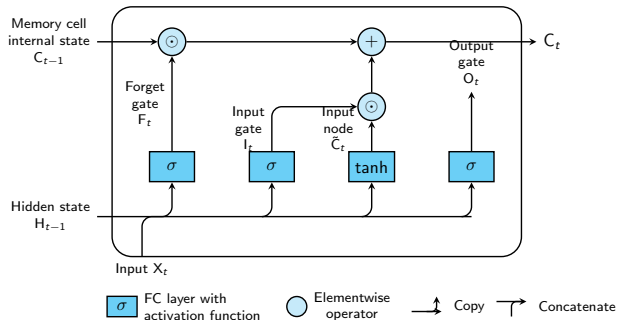
**Candidate** (dùng tanh, phạm vi  $(-1, 1)$ ):

$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c)$$

**Tham số:**

- $W_{xc} \in \mathbb{R}^{d \times h}$ : trọng số input
- $W_{hc} \in \mathbb{R}^{h \times h}$ : trọng số hidden
- $b_c \in \mathbb{R}^{1 \times h}$  (bias)

# Updating the Memory Cell Internal State



Cập nhật ô nhớ dùng tích Hadamard  $\odot$ :

Vai trò các cổng:

- $I_t$ : kiểm soát lượng thông tin **mới** ( $\tilde{C}_t$ ) được ghi vào
- $F_t$ : kiểm soát lượng thông tin **cũ** ( $C_{t-1}$ ) được giữ lại

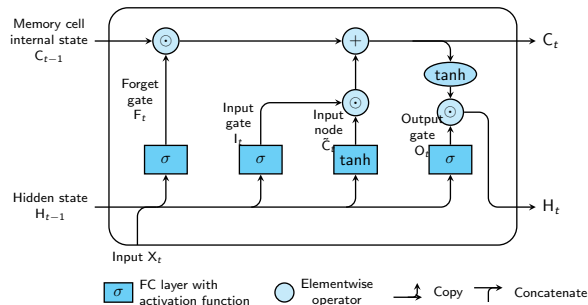
Nhận xét:

- $F_t = 1, I_t = 0$ :  $C_t = C_{t-1}$  (giữ nguyên)
- Mô hình tự học **khi nào giữ**, khi nào cập nhật
- Giúp **giảm vanishing gradient**, dễ huấn luyện trên chuỗi dài

Phương trình cập nhật:

$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$$

# Tính toán Hidden State (Đầu ra của ô nhớ)



## Phương trình:

$$H_t = O_t \odot \tanh(C_t)$$

**Tính Hidden State**  $H_t \in \mathbb{R}^{n \times h}$  — đầu ra của ô nhớ:

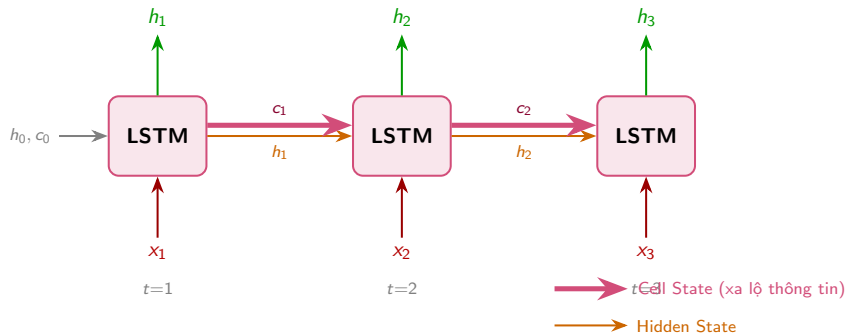
### Cơ chế:

- Áp dụng  $\tanh$  lên cell state  $C_t$
- Nhân element-wise với **Output gate**  $O_t$

### Nhận xét:

- $H_t$  luôn nằm trong  $(-1, 1)$
- $O_t \approx 1$ : cell state tác động trực tiếp lên các lớp tiếp theo
- $O_t \approx 0$ : thông tin được **tích lũy âm thầm** trong  $C_t$  mà không ảnh hưởng mạng
- Khi  $O_t$  chuyển từ  $0 \rightarrow 1$ : thông tin **bùng phát** ra ngoài

# Kiến trúc LSTM — Dạng mở (Unrolled) theo thời gian



## Hai “dòng chảy” song song

- **Cell State**  $c_t$ : Xả lộ thông tin — thay đổi chậm, giữ thông tin dài hạn
- **Hidden State**  $h_t$ : Bộ nhớ làm việc — thay đổi nhanh, dùng cho output

# Ba cổng của LSTM — Giải thích trực giác

Quay lại bài toán đoán từ: “Tôi sinh ra ở **Việt Nam**. Sau đó tôi sang **Pháp** học. Tiếng mẹ đẻ là tiếng \_\_”

## 1. Forget Gate ( $f_t$ )

Quên

**Hỏi:** “Nên quên gì?”

Khi gặp “sang Pháp học”: không xóa “Việt Nam” vì nó quan trọng cho “tiếng mẹ đẻ”

## 2. Input Gate ( $i_t$ )

Ghi

**Hỏi:** “Nên ghi gì?”

Khi gặp “Pháp”: ghi vào “nơi du học” nhưng không ghi đề “nơi sinh”

## 3. Output Gate ( $o_t$ )

Đọc

**Hỏi:** “Nên xuất gì?”

Khi cần đoán “tiếng mẹ đẻ”: đọc lại thông tin “nơi sinh”  $\Rightarrow$  Việt

**Tất cả cổng** đều dùng hàm Sigmoid  $\sigma(\cdot) \in (0, 1)$ :  
 $0 =$  đóng hoàn toàn (chặn),  $1 =$  mở hoàn toàn (cho qua)

# Công thức toán cốt lõi của LSTM

Bây giờ ta đã hiểu trực giác, hãy xem công thức:

**Bước 1 — Forget Gate:** Quyết định quên gì từ  $c_{t-1}$

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

**Bước 2 — Input Gate + Candidate:** Quyết định ghi gì mới

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i), \quad \tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

**Bước 3 — Cập nhật Cell State:** Quên cũ + ghi mới

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

**Bước 4 — Output Gate:** Quyết định xuất gì

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(c_t)$$

$\odot$  = phép nhân theo phần tử (element-wise),  $[\cdot, \cdot]$  = nối vector (concatenate)

# Ví dụ tính tay: Thiết lập bài toán

## Thiết lập

- **Vocab** = {A, B, C}  $\Rightarrow$  vocab size = 3
- **Hidden size** = 2,    **Sequence**:  $x_1 = A, x_2 = B, x_3 = C$
- One-hot encoding:  $A = [1, 0, 0]^T, B = [0, 1, 0]^T, C = [0, 0, 1]^T$

## Khởi tạo

$$h_0 = [0, 0]^T, \quad c_0 = [0, 0]^T$$

## Trọng số (đơn giản hóa để tính tay)

Gộp  $[h_{t-1}, x_t]$  thành vector 5 chiều. Mỗi cổng có  $W \in \mathbb{R}^{2 \times 5}, b \in \mathbb{R}^2$ :

$$W_f = W_i = W_o = W_c = \begin{bmatrix} 0.1 & 0.1 & 0.2 & 0.1 & 0.0 \\ 0.0 & 0.1 & 0.1 & 0.0 & 0.2 \end{bmatrix}, \quad b_f = b_i = b_o = b_c = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

(Dùng cùng trọng số cho đơn giản — thực tế mỗi cổng có trọng số riêng)



Ví dụ tính tay: Bước  $t = 1$  ( $x_1 = A = [1, 0, 0]^T$ )

Nội vector:  $[h_0, x_1] = [0, 0, 1, 0, 0]^T$

Tính tích  $W \cdot [h_0, x_1]$ : 
$$\begin{bmatrix} 0.1 & 0.1 & 0.2 & 0.1 & 0.0 \\ 0.0 & 0.1 & 0.1 & 0.0 & 0.2 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 0.1 \end{bmatrix}$$

**Forget Gate:**

$$f_1 = \sigma([0.2, 0.1]) = [0.550, 0.525]$$

**Candidate:**

$$\tilde{c}_1 = \tanh([0.2, 0.1]) = [0.197, 0.100]$$

**Input Gate:**

$$i_1 = \sigma([0.2, 0.1]) = [0.550, 0.525]$$

**Output Gate:**

$$o_1 = \sigma([0.2, 0.1]) = [0.550, 0.525]$$

**Cập nhật Cell State:**  $c_1 = f_1 \odot c_0 + i_1 \odot \tilde{c}_1 = [0, 0] + [0.550 \times 0.197, 0.525 \times 0.100] = [0.108, 0.052]$

**Hidden State:**  $h_1 = o_1 \odot \tanh(c_1) = [0.550 \times 0.108, 0.525 \times 0.052] = [0.059, 0.027]$

Ví dụ tính tay: Bước  $t = 2$  ( $x_2 = B = [0, 1, 0]^T$ )

Nội vector:  $[h_1, x_2] = [0.059, 0.027, 0, 1, 0]^T$

Tính tích  $W \cdot [h_1, x_2]$ :  $\begin{bmatrix} 0.1(0.059) + 0.1(0.027) + 0.2(0) + 0.1(1) + 0.0(0) \\ 0.0(0.059) + 0.1(0.027) + 0.1(0) + 0.0(1) + 0.2(0) \end{bmatrix} = \begin{bmatrix} 0.109 \\ 0.003 \end{bmatrix}$

**Forget:**  $f_2 = \sigma([0.109, 0.003])$   
 $= [0.527, 0.501]$

**Input:**  $i_2 = [0.527, 0.501]$

**Candidate:**  $\tilde{c}_2 = \tanh([0.109, 0.003])$   
 $= [0.109, 0.003]$

**Output:**  $o_2 = [0.527, 0.501]$

**Cell State:**

$c_2 = f_2 \odot c_1 + i_2 \odot \tilde{c}_2$   
 $= [0.527 \times 0.108, 0.501 \times 0.052]$   
 $+ [0.527 \times 0.109, 0.501 \times 0.003]$   
 $= [0.057 + 0.057, 0.026 + 0.002]$   
 $= [0.114, 0.028]$

**Hidden State:**  $h_2 = o_2 \odot \tanh(c_2) = [0.527 \times 0.114, 0.501 \times 0.028] = [0.060, 0.014]$

Nhận xét:  $c_2 \approx c_1$  — Cell State thay đổi từ từ, giữ thông tin ổn định!

Ví dụ tính tay: Bước  $t = 3$  ( $x_3 = C = [0, 0, 1]^T$ )

Nội vector:  $[h_2, x_3] = [0.060, 0.014, 0, 0, 1]^T$

Tính tích:  $W \cdot [h_2, x_3] = \begin{bmatrix} 0.1(0.060) + 0.1(0.014) + 0 + 0 + 0 \\ 0 + 0.1(0.014) + 0 + 0 + 0.2(1) \end{bmatrix} = \begin{bmatrix} 0.007 \\ 0.201 \end{bmatrix}$

$f_3 = \sigma([0.007, 0.201]) = [0.502, 0.550]$ ,  $i_3 = [0.502, 0.550]$

$\tilde{c}_3 = \tanh([0.007, 0.201]) = [0.007, 0.199]$ ,  $o_3 = [0.502, 0.550]$

Cell State:  $c_3 = [0.502 \times 0.114, 0.550 \times 0.028] + [0.502 \times 0.007, 0.550 \times 0.199] = [0.057 + 0.004, 0.015 + 0.109] = [0.061, 0.125]$

Hidden State:  $h_3 = o_3 \odot \tanh(c_3) = [0.502 \times 0.061, 0.550 \times 0.124] = [0.031, 0.068]$

Tổng kết 3 bước

| $t$ | $c_t[1]$ | $c_t[2]$ | $h_t[1]$ | $h_t[2]$ |
|-----|----------|----------|----------|----------|
| 1   | 0.108    | 0.052    | 0.059    | 0.027    |
| 2   | 0.114    | 0.028    | 0.060    | 0.014    |
| 3   | 0.061    | 0.125    | 0.031    | 0.068    |

# Tổng kết: So sánh RNN và LSTM

| RNN                                                                                         |            | LSTM                                    |
|---------------------------------------------------------------------------------------------|------------|-----------------------------------------|
| 1 trạng thái: $h_t$                                                                         | Trạng thái | 2 trạng thái: $h_t$ và $c_t$            |
| Không có cơ chế cổng                                                                        | Cơ chế     | 3 cổng: Forget, Input, Output           |
| Gradient dễ vanish                                                                          | Gradient   | Gradient ổn định hơn                    |
| Nhớ ngắn hạn                                                                                | Trí nhớ    | Nhớ dài hạn + ngắn hạn                  |
| Ít tham số, nhanh                                                                           | Chi phí    | Nhiều tham số hơn ( $\approx 4\times$ ) |
| Chuỗi ngắn, đơn giản                                                                        | Phù hợp    | Chuỗi dài, phức tạp                     |
| <b>LSTM = RNN + Cơ chế cổng + Cell State</b><br>$\Rightarrow$ Giải quyết vanishing gradient |            |                                         |

# Tóm tắt bài học

- ❶ **Vấn đề:** RNN gặp vanishing gradient  $\Rightarrow$  không nhớ được phụ thuộc xa
- ❷ **Giải pháp:** LSTM thêm *Cell State* (xa lộ thông tin) và 3 cổng để kiểm soát dòng chảy thông tin
- ❸ **Ba cổng:**
  - ▶ Forget Gate — quyết định quên gì
  - ▶ Input Gate — quyết định ghi gì mới
  - ▶ Output Gate — quyết định xuất gì
- ❹ **Kết quả:** Gradient chảy ổn định qua Cell State  $\Rightarrow$  học được phụ thuộc dài hạn

## Bước tiếp theo

- GRU (Gated Recurrent Unit) — phiên bản rút gọn của LSTM
- Bidirectional LSTM — đọc chuỗi cả hai chiều
- Attention Mechanism  $\rightarrow$  Transformer

# Gated Recurrent Unit (GRU)

A simpler alternative to LSTM

Can we achieve similar performance with fewer parameters?

## GRU. Bài toán: Đoán từ tiếp theo

**Thử sức nào!** Hãy đoán từ còn thiếu:

- ① “Mặt trời mọc ở hướng \_\_\_\_\_” → **Đông**
- ② “Anh ấy rất giỏi toán nên đã đậu \_\_\_\_\_” → **đại học**
- ③ “Cô ấy sinh ra ở Nhật Bản. Sau nhiều năm sống ở Việt Nam, cô ấy vẫn nói thành thạo tiếng \_\_\_\_\_” → **Nhật**

### Nhận xét

- Câu 1–2: Chỉ cần ngữ cảnh **gần** là đoán được
- Câu 3: Cần nhớ thông tin từ **rất xa** (“Nhật Bản”)
- RNN gặp khó khăn với câu 3 ⇒ LSTM ra đời ⇒ nhưng LSTM có **quá phức tạp** không?

# Nhắc lại: RNN và LSTM — Ưu nhược điểm

## RNN — Quá đơn giản

- Chỉ 1 trạng thái:  $h_t$
- Không có cơ chế cổng
- Gradient vanish  $\Rightarrow$  quên nhanh
- ✓ Ít tham số, nhanh
- ✗ Không nhớ xa

## LSTM — Quá phức tạp?

- 2 trạng thái:  $h_t$  và  $c_t$
- 3 cổng: Forget, Input, Output
- Giải quyết vanishing gradient
- ✓ Nhớ xa tốt
- ✗ **Nhiều tham số** ( $\approx 4 \times$  RNN)
- ✗ **Huấn luyện chậm**

**Câu hỏi:** Có thể thiết kế mô hình nhớ xa như LSTM nhưng **đơn giản hơn, ít tham số hơn, nhanh hơn** không?



# Ý tưởng: Đơn giản hóa LSTM $\Rightarrow$ GRU

**GRU** (Cho et al., 2014) đơn giản hóa LSTM bằng 2 thay đổi lớn:

## Thay đổi 1: Gộp trạng thái



Gộp  $c_t$  và  $h_t$  thành **1 trạng thái duy nhất**  $h_t$

## Thay đổi 2: Gộp cổng



Gộp Forget + Input  $\rightarrow$  **Update gate**  
Thay Output  $\rightarrow$  **Reset gate**

**Kết quả:** GRU có ít tham số hơn LSTM  $\approx 25\%$ , huấn luyện nhanh hơn, mà hiệu quả tương đương trên nhiều bài toán.

# Hai cổng của GRU — Giải thích trực giác

**Phép ẩn dụ:** Bạn đang đọc tin tức và cập nhật kiến thức.

## Update Gate ( $z_t$ )

Update

**“Giữ bao nhiêu % cũ,  
lấy bao nhiêu % mới?”**

$z_t \approx 1$ : Giữ nguyên kiến thức cũ

$z_t \approx 0$ : Thay hoàn toàn bằng mới

*Ví dụ:* Đang đọc về thể thao, gặp tin thời tiết  $\rightarrow z_t$  cao (giữ kiến thức thể thao)

## Reset Gate ( $r_t$ )

Reset

**“Tham khảo quá khứ  
bao nhiêu khi tạo ứng viên?”**

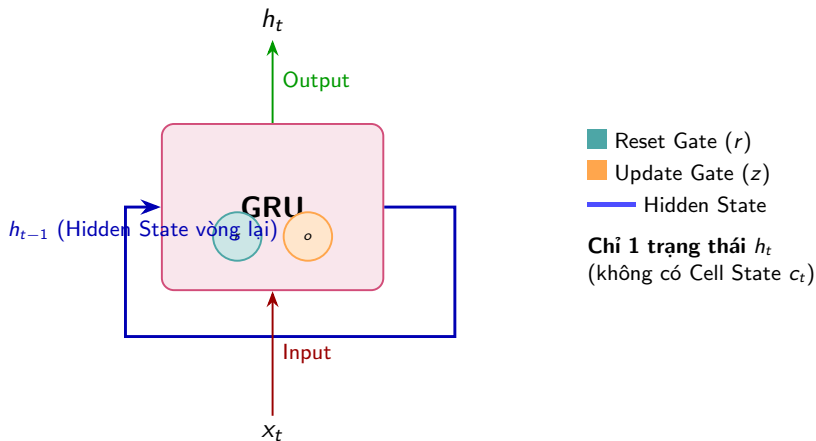
$r_t \approx 1$ : Dùng đầy đủ quá khứ

$r_t \approx 0$ : Bỏ qua, bắt đầu mới

*Ví dụ:* Chuyển đề tài hoàn toàn  $\rightarrow r_t$  thấp (quên quá khứ khi tạo ứng viên)

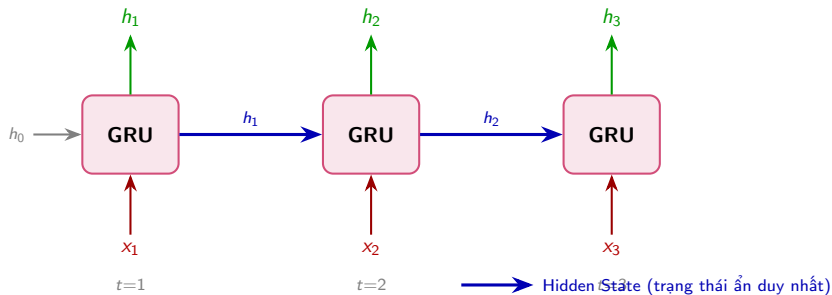
**Cả hai cổng** đều dùng hàm Sigmoid  $\sigma(\cdot) \in (0, 1)$   
 $0$  = đóng hoàn toàn,  $1$  = mở hoàn toàn

# Kiến trúc GRU — Dạng cuộn (Rolled)



- GRU chỉ truyền **1 trạng thái**  $h_t$  qua thời gian (vs. LSTM:  $h_t + c_t$ )
- Đơn giản hơn nhưng vẫn có cơ chế cổng để kiểm soát dòng thông tin

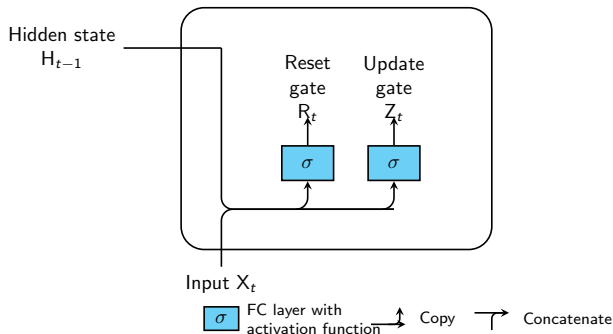
# Kiến trúc GRU — Dạng mở (Unrolled) theo thời gian



## So sánh nhanh với LSTM

- LSTM truyền **2 đường**: Cell State  $c_t$  + Hidden State  $h_t$
- GRU truyền **1 đường** duy nhất: Hidden State  $h_t$
- $\Rightarrow$  Ít phép tính hơn, nhanh hơn mỗi bước

# Gated Recurrent Unit (GRU) Architecture



**Tính toán cổng** (dùng  $\sigma$ , phạm vi  $(0, 1)$ ):

$$R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r)$$

$$Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$

Kiến trúc GRU thay thế 3 cổng của LSTM bằng 2 cổng: **cổng thiết lập lại** (reset gate)  $R_t$  và **cổng cập nhật** (update gate)  $Z_t$ .

**Ý nghĩa:**

- **Reset gate:** Kiểm soát lượng thông tin trạng thái quá khứ mà chúng ta vẫn muốn giữ lại.
- **Update gate:** Kiểm soát mức độ trạng thái mới chỉ là bản sao của trạng thái cũ.

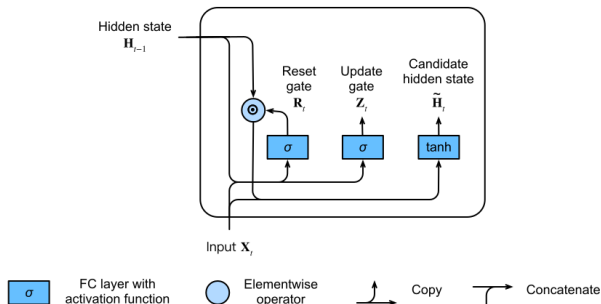
**Đầu vào:**

- $X_t \in \mathbb{R}^{n \times d}$ : input tại bước  $t$
- $H_{t-1} \in \mathbb{R}^{n \times h}$ : hidden state bước trước

**Tham số:**

- $W_{xr}, W_{xz} \in \mathbb{R}^{d \times h}$ : trọng số input
- $W_{hr}, W_{hz} \in \mathbb{R}^{h \times h}$ : trọng số hidden
- $b_r, b_z \in \mathbb{R}^{1 \times h}$  (bias)

# Gated Recurrent Unit (GRU) - Final Update



Tính toán **trạng thái ẩn ứng viên** (candidate hidden state): tích hợp **cổng thiết lập lại**  $R_t$  để tạo ra  $\tilde{H}_t \in \mathbb{R}^{n \times h}$ .

**Phương trình** (dùng tanh và phép nhân từng phần tử  $\odot$ ):

$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$

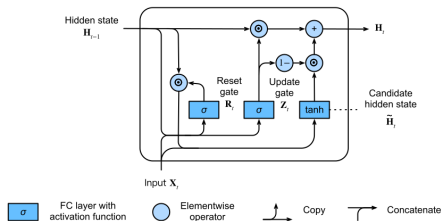
**Tham số:**

- $W_{xh} \in \mathbb{R}^{d \times h}$ : trọng số input
- $W_{hh} \in \mathbb{R}^{h \times h}$ : trọng số hidden
- $b_h \in \mathbb{R}^{1 \times h}$  (bias)

## Cơ chế hoạt động:

- Khi  $R_t \approx 1$ : Phục hồi mạng RNN tiêu chuẩn (giữ nguyên ảnh hưởng của trạng thái trước  $H_{t-1}$ ).
- Khi  $R_t \approx 0$ : Trạng thái cũ bị reset,  $\tilde{H}_t$  trở thành kết quả của một MLP chỉ phụ thuộc vào đầu vào  $X_t$ .

# Gated Recurrent Unit (GRU) - Final Update



**Cập nhật cuối cùng** (tổ hợp lồi theo từng phần tử):

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$

Cập nhật **trạng thái ẩn** (hidden state): sử dụng **cổng cập nhật**  $Z_t$  để xác định trạng thái mới  $H_t \in \mathbb{R}^{n \times h}$ .

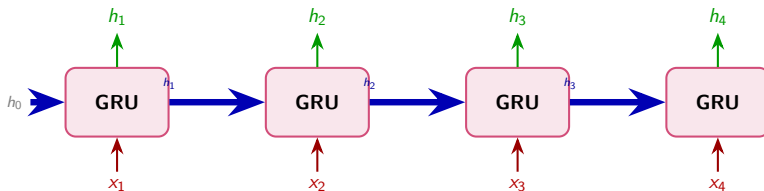
**Thành phần:**

- $H_{t-1} \in \mathbb{R}^{n \times h}$ : hidden state bước trước
- $\tilde{H}_t \in \mathbb{R}^{n \times h}$ : trạng thái ứng viên mới (candidate)
- $Z_t \in \mathbb{R}^{n \times h}$ : cổng cập nhật (update gate)

**Cơ chế hoạt động:**

- Khi  $Z_t \approx 1$ : Giữ lại trạng thái cũ  $H_{t-1}$ , bỏ qua  $X_t$  (nhảy bước).
- Khi  $Z_t \approx 0$ : Trạng thái mới  $H_t$  tiến gần đến ứng viên  $\tilde{H}_t$ .

# Dòng chảy Hidden State qua các bước thời gian



**Tại mỗi bước:** Update gate  $z_t$  quyết định tỷ lệ trộn giữa  $h_{t-1}$  (cũ) và  $\tilde{h}_t$  (mới):  
$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$$

Khi  $z_t \rightarrow 1$ :  $h_t \approx h_{t-1}$  (giữ nguyên, thông tin chảy thẳng qua  $\Rightarrow$  nhớ xa)

Khi  $z_t \rightarrow 0$ :  $h_t \approx \tilde{h}_t$  (cập nhật hoàn toàn, quên quá khứ)



# Công thức toán cốt lõi của GRU

Bây giờ ta đã hiểu trực giác, hãy xem công thức:

**Bước 1 — Update Gate:** Tỷ lệ giữ cũ vs. lấy mới

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

**Bước 2 — Reset Gate:** Mức độ tham khảo quá khứ khi tạo ứng viên

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

**Bước 3 — Candidate Hidden State:** Đề xuất trạng thái mới

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h)$$

**Bước 4 — Cập nhật Hidden State:** Trộn cũ và mới

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$$

$\odot$  = nhân theo phần tử,  $[\cdot, \cdot]$  = nối vector (concatenate)

**Lưu ý:** Bước 4 dùng  $z_t$  cho cả “giữ cũ” và “lấy mới” (bổ sung nhau:  $z + (1-z) = 1$ ).  
LSTM cần 2 cổng riêng (Forget + Input), GRU chỉ cần 1 cổng Update!

# Ví dụ tính tay: Thiết lập bài toán

## Thiết lập

- **Vocab** = {A, B, C}  $\Rightarrow$  vocab size = 3
- **Hidden size** = 2,    **Sequence**:  $x_1 = A, x_2 = B, x_3 = C$
- One-hot:  $A = [1, 0, 0]^T, B = [0, 1, 0]^T, C = [0, 0, 1]^T$

## Khởi tạo

$$h_0 = [0, 0]^T$$

## Trọng số (đơn giản hóa)

Gộp  $[h_{t-1}, x_t]$  thành vector 5 chiều. Mỗi cổng có  $W \in \mathbb{R}^{2 \times 5}, b \in \mathbb{R}^2$ :

$$W_z = W_r = W_h = \begin{bmatrix} 0.1 & 0.1 & 0.2 & 0.1 & 0.0 \\ 0.0 & 0.1 & 0.1 & 0.0 & 0.2 \end{bmatrix}, \quad b_z = b_r = b_h = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

(Dùng cùng trọng số cho đơn giản — thực tế mỗi cổng có trọng số riêng)

**Nhắc lại:**  $\sigma(0.2) \approx 0.550, \sigma(0.1) \approx 0.525, \tanh(0.2) \approx 0.197, \tanh(0.1) \approx 0.100$

Ví dụ tính tay: Bước  $t = 1$  ( $x_1 = A = [1, 0, 0]^T$ )

Nội vector:  $[h_0, x_1] = [0, 0, 1, 0, 0]^T$

Tích:  $W \cdot [h_0, x_1] = [0.2, 0.1]^T$  (cột thứ 3 của  $W$ )

**Update Gate:**

$$z_1 = \sigma([0.2, 0.1]) = [0.550, 0.525]$$

**Reset Gate:**

$$r_1 = \sigma([0.2, 0.1]) = [0.550, 0.525]$$

**Candidate:**

$$r_1 \odot h_0 = [0, 0] \text{ (vì } h_0 = 0)$$

$$[r_1 \odot h_0, x_1] = [0, 0, 1, 0, 0]^T$$

$$\tilde{h}_1 = \tanh([0.2, 0.1]) = [0.197, 0.100]$$

**Cập nhật Hidden State:**

$$\begin{aligned} h_1 &= z_1 \odot h_0 + (1 - z_1) \odot \tilde{h}_1 \\ &= [0.550, 0.525] \odot [0, 0] + [0.450, 0.475] \odot [0.197, 0.100] \\ &= [0, 0] + [0.089, 0.047] \\ &= [0.089, 0.047] \end{aligned}$$

Ví dụ tính tay: Bước  $t = 2$  ( $x_2 = B = [0, 1, 0]^T$ )

**Nội vector:**  $[h_1, x_2] = [0.089, 0.047, 0, 1, 0]^T$

**Tích:**  $W \cdot [h_1, x_2] = [0.1(0.089) + 0.1(0.047) + 0.1(1), 0.1(0.047) + 0(1)] = [0.114, 0.005]$

**Update:**  $z_2 = \sigma([0.114, 0.005]) = [0.528, 0.501]$

**Reset:**  $r_2 = [0.528, 0.501]$

**Candidate:**  $r_2 \odot h_1 = [0.528 \times 0.089, 0.501 \times 0.047] = [0.047, 0.024]$

$W \cdot [0.047, 0.024, 0, 1, 0]^T = [0.1(0.047) + 0.1(0.024) + 0.1(1), 0.1(0.024)] = [0.107, 0.002]$

$\tilde{h}_2 = \tanh([0.107, 0.002]) = [0.107, 0.002]$

**Cập nhật:**  $h_2 = [0.528, 0.501] \odot [0.089, 0.047] + [0.472, 0.499] \odot [0.107, 0.002]$   
 $= [0.047, 0.024] + [0.051, 0.001] = [0.098, 0.025]$

Ví dụ tính tay: Bước  $t = 3$  ( $x_3 = C = [0, 0, 1]^T$ )

**Nội vector:**  $[h_2, x_3] = [0.098, 0.025, 0, 0, 1]^T$

**Tích:**  $W \cdot [h_2, x_3] = [0.1(0.098) + 0.1(0.025) + 0, 0.1(0.025) + 0.2(1)] = [0.012, 0.203]$

$z_3 = \sigma([0.012, 0.203]) = [0.503, 0.551], \quad r_3 = [0.503, 0.551]$

**Candidate:**  $r_3 \odot h_2 = [0.503 \times 0.098, 0.551 \times 0.025] = [0.049, 0.014]$

$W \cdot [0.049, 0.014, 0, 0, 1]^T = [0.006, 0.201]$

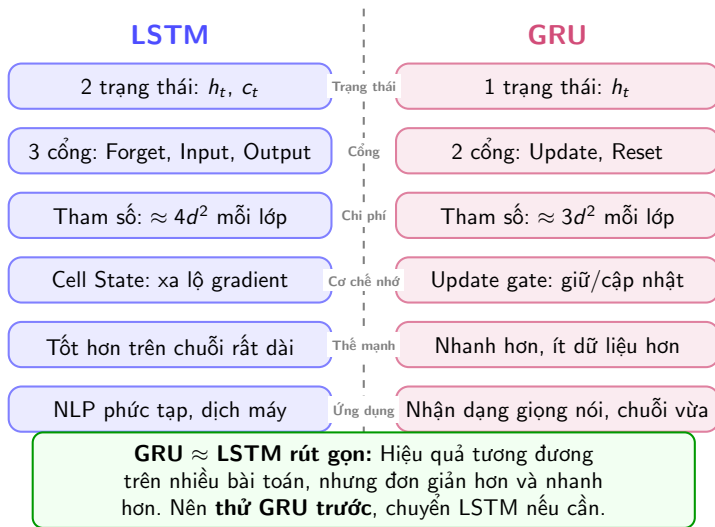
$\tilde{h}_3 = \tanh([0.006, 0.201]) = [0.006, 0.199]$

$h_3 = [0.503, 0.551] \odot [0.098, 0.025] + [0.497, 0.449] \odot [0.006, 0.199]$   
 $= [0.049, 0.014] + [0.003, 0.089] = [0.052, 0.103]$

Tổng kết 3 bước

| $t$ | $z_t[1]$ | $z_t[2]$ | $\tilde{h}_t[1]$ | $\tilde{h}_t[2]$ | $h_t[1]$ | $h_t[2]$ |
|-----|----------|----------|------------------|------------------|----------|----------|
| 1   | 0.550    | 0.525    | 0.197            | 0.100            | 0.089    | 0.047    |
| 2   | 0.528    | 0.501    | 0.107            | 0.002            | 0.098    | 0.025    |
| 3   | 0.503    | 0.551    | 0.006            | 0.199            | 0.052    | 0.103    |

# Tổng kết: So sánh LSTM và GRU



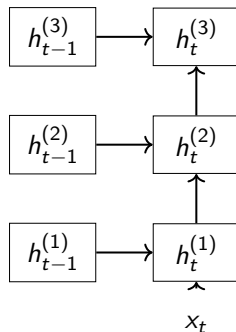
# Tóm tắt bài học

- ❶ **Động lực:** LSTM mạnh nhưng phức tạp (3 cổng, 2 trạng thái)
- ❷ **GRU đơn giản hóa bằng cách:**
  - ▶ Gộp Cell State và Hidden State → **1 trạng thái**  $h_t$
  - ▶ Gộp Forget + Input gate → **Update gate**  $z_t$
  - ▶ Thay Output gate → **Reset gate**  $r_t$
- ❸ **Công thức cốt lõi:**  $h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$
- ❹ **Kết quả:** Ít hơn  $\approx 25\%$  tham số, nhanh hơn, hiệu quả tương đương

## Bước tiếp theo

- So sánh thực nghiệm LSTM vs GRU trên bài toán cụ thể
- Bidirectional GRU — đọc chuỗi cả hai chiều
- Attention Mechanism → Transformer

# Deep RNN Architecture

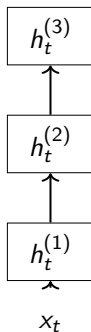


- Nhiều tầng RNN xếp chồng lên nhau
- Mỗi tầng học một mức biểu diễn khác nhau
- Tăng khả năng học quan hệ phức tạp



# Deep Recurrent Neural Networks

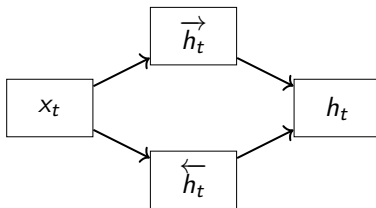
- Deep RNN = nhiều tầng RNN xếp chồng lên nhau
- Mỗi tầng học một mức biểu diễn khác nhau
- Tăng khả năng học quan hệ phức tạp trong chuỗi



- Nhược điểm: khó train, dễ overfitting, tốn tài nguyên

# Bidirectional Recurrent Neural Networks

- BiRNN = hai RNN chạy theo hai chiều của chuỗi
- Forward: trái  $\rightarrow$  phải
- Backward: phải  $\rightarrow$  trái



- Kết hợp thông tin từ quá khứ và tương lai
- Hiệu quả trong NLP (sentiment, NER, ...)

# The Encoder–Decoder Architecture

Kiến trúc Mã hóa – Giải mã

Tiền đề: Feedforward NN, RNN, LSTM, GRU

# Bài toán: Dịch máy (Machine Translation)

Thử dịch câu sau:

“Tôi yêu Việt Nam”  $\longrightarrow$  “I love Vietnam”

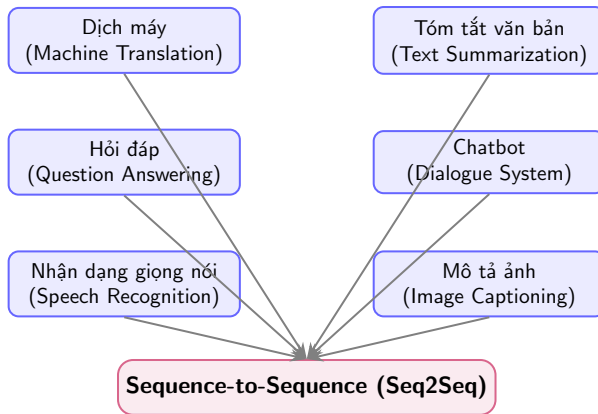
## Quan sát

- Câu đầu vào có **3 từ**, câu đầu ra có **3 từ** — nhưng không phải lúc nào cũng vậy!
- “Tôi đang đi học” (4 từ)  $\rightarrow$  “I am going to school” (**5 từ**)
- Đầu vào và đầu ra có **độ dài khác nhau, ngôn ngữ khác nhau**

## Vấn đề

RNN/LSTM/GRU nhận chuỗi  $\rightarrow$  xuất chuỗi **cùng độ dài**.  
Làm sao xử lý khi đầu vào và đầu ra có **độ dài khác nhau**?

# Không chỉ dịch máy — Rất nhiều bài toán Seq2Seq



**Đặc điểm chung:** Đầu vào là một chuỗi, đầu ra là một chuỗi khác, hai chuỗi có thể **khác độ dài, khác miền** (ngôn ngữ, âm thanh, ảnh...).

# Ý tưởng: Chia bài toán thành 2 giai đoạn

**Phép ẩn dụ:** Phiên dịch viên làm việc thế nào?

## Bước 1: Nghe & Hiểu

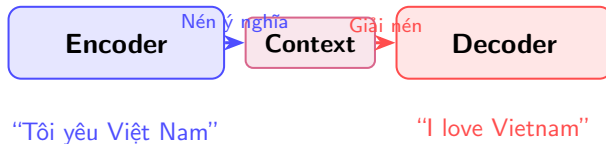
- Nghe toàn bộ câu tiếng Việt
- Hiểu **ý nghĩa tổng thể**
- Ghi nhớ trong đầu → **“bản tóm tắt ý nghĩa”**

⇒ **Encoder** (Mã hóa)

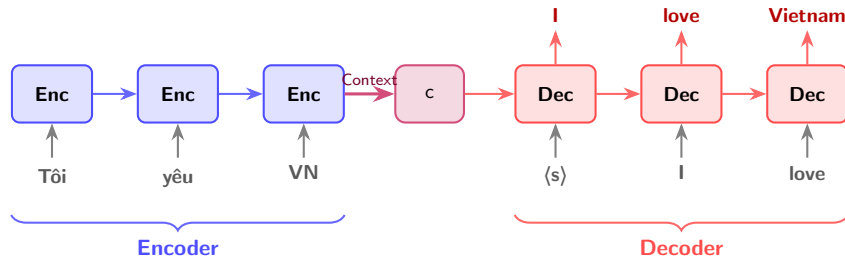
## Bước 2: Diễn đạt

- Dựa trên bản tóm tắt ý nghĩa
- Nói lại bằng tiếng Anh
- Nói **từng từ một**

⇒ **Decoder** (Giải mã)



# Kiến trúc Encoder–Decoder: Tổng quan



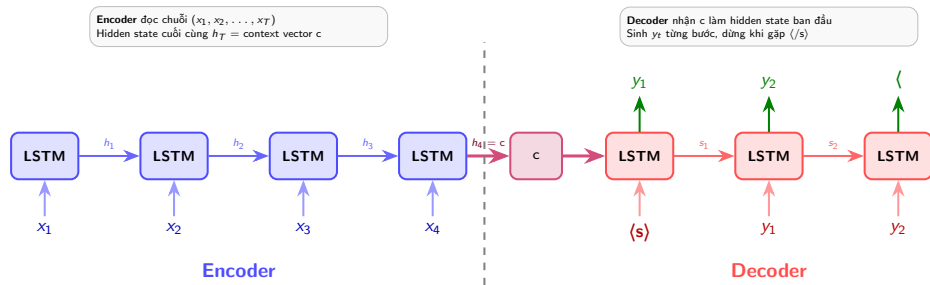
## Encoder

- Đọc toàn bộ chuỗi đầu vào
- Nén thành **context vector**  $c$
- Thường dùng RNN/LSTM/GRU

## Decoder

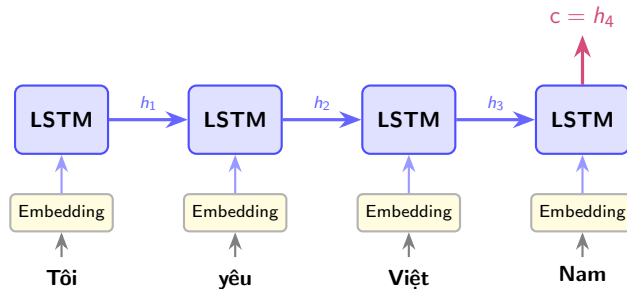
- Nhận context vector  $c$
- Sinh chuỗi đầu ra **từng bước**
- Đầu ra bước  $t$  là đầu vào bước  $t+1$

# Dòng chảy dữ liệu (Data Flow) chi tiết



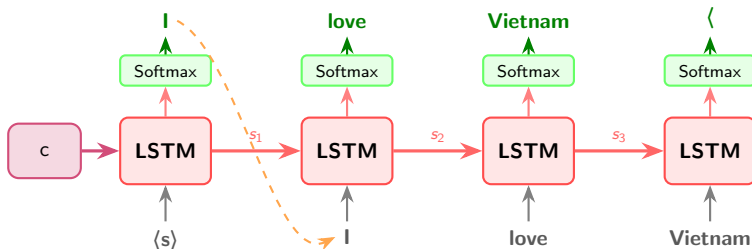


## Encoder: Mã hóa chuỗi đầu vào



**Encoder** đọc lần lượt từng từ qua Embedding  $\rightarrow$  LSTM.  
Hidden state cuối cùng  $h_T$  chứa thông tin toàn bộ câu  $\rightarrow$  context vector  $c$ .

## Decoder: Giải mã từng bước



**Decoder** nhận  $c$  làm  $s_0$ , bắt đầu bằng token  $\langle s \rangle$ . Mỗi bước: LSTM  $\rightarrow$  Softmax  $\rightarrow$  từ tiếp theo. Dừng khi sinh token  $\langle /s \rangle$ . *Teacher forcing*: khi huấn luyện, dùng từ đúng (ground truth) làm đầu vào.

# Công thức toán cốt lõi

Bây giờ ta đã hiểu trực giác, hãy xem công thức:

**Encoder:** Đọc chuỗi  $(x_1, x_2, \dots, x_T)$ , cập nhật hidden state:

$$h_t = f_{\text{enc}}(x_t, h_{t-1}) \quad t = 1, 2, \dots, T$$

Context vector:  $c = h_T$  (hoặc  $c = q(h_1, h_2, \dots, h_T)$ )

**Decoder:** Sinh chuỗi  $(y_1, y_2, \dots, y_{T'})$  từng bước:

$$s_t = f_{\text{dec}}(y_{t-1}, s_{t-1}, c) \quad t = 1, 2, \dots, T'$$

Phân phối đầu ra:  $P(y_t \mid y_1, \dots, y_{t-1}, c) = g(s_t)$  (Softmax)

**Hàm mất mát:** Tối đa hóa log-likelihood:

$$\mathcal{L} = - \sum_{t=1}^{T'} \log P(y_t^* \mid y_1^*, \dots, y_{t-1}^*, c)$$

với  $y_t^*$  là từ đúng (ground truth) tại bước  $t$ .

$f_{\text{enc}}, f_{\text{dec}}$ : hàm RNN/LSTM/GRU.  $T$ : độ dài đầu vào.  $T'$ : độ dài đầu ra (có thể  $\neq T$ ).

# Hạn chế của Encoder–Decoder cơ bản



## Vấn đề nghẽn cổ chai (Information Bottleneck)

- **Toàn bộ** thông tin câu đầu vào phải nén vào **1 vector** c duy nhất
- Câu càng dài → c càng khó chứa đủ thông tin → **mất mát thông tin**
- Decoder ở các bước sau càng xa c → càng “quên” ngữ cảnh

## Giải pháp → Attention Mechanism (bài tiếp theo)

- Thay vì 1 vector c cố định, cho Decoder “nhìn lại” **tất cả** hidden states của Encoder
- Mỗi bước giải mã, Decoder tự chọn phần nào của đầu vào cần “chú ý” nhất

# Tóm tắt

- ➊ **Vấn đề:** Nhiều bài toán cần ánh xạ chuỗi  $\rightarrow$  chuỗi với **độ dài khác nhau**
- ➋ **Giải pháp:** Kiến trúc **Encoder–Decoder**
  - ▶ **Encoder:** đọc chuỗi đầu vào, nén thành context vector  $c$
  - ▶ **Decoder:** từ  $c$ , sinh chuỗi đầu ra từng bước
- ➌ **Ưu điểm:** Xử lý chuỗi đầu vào/đầu ra khác độ dài, khác miền
- ➍ **Hạn chế:** Nghẽn cổ chai — 1 vector  $c$  khó chứa đủ thông tin câu dài

## Bước tiếp theo

- **Attention Mechanism** — cho phép Decoder “nhìn lại” toàn bộ Encoder
- **Transformer** — thay thế RNN hoàn toàn bằng Self-Attention

Questions?